

Day 10 lab: small language models on the Raspberry Pi

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 3, 2026

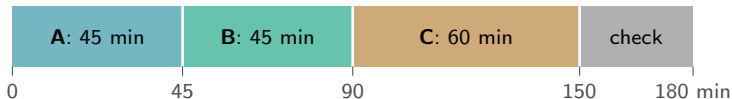
- ▶ **Goal:** a measured comparison of two small language models on the Raspberry Pi 5, and one working application.
- ▶ **Board:** the Raspberry Pi 5 (8 GB) with the active cooler.
- ▶ **Deliverable:** the file `report.md` with the benchmark table, the results of the application, and the Decision Log.

From the lecture to the lab

The lecture gave the RAM budget, the two rates, the options of Ollama, and the methods that connect a model to a program. Its rates came from the sources and from an x86 computer. Today you measure the board, and you check the output of each model with a schema and with your code.

Key point: A rate needs its method. An output needs a check: valid is not the same as correct.

Plan of the three hours



Part	Work	Time
A	Run and measure: two models, tokens per second, RAM, and temperature	45 min
B	Python integration: structured output with a schema, and a function call	45 min
C	Application: retrieval-augmented generation or image description	60 min
Check by the instructor		30 min

Write each result in `report.md` when you get it.

Hardware for each group

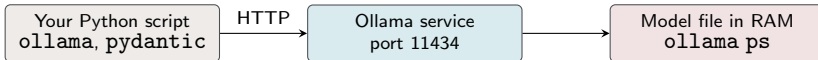
- ▶ Raspberry Pi 5 (8 GB) with the active cooler, the card of your group, and the 27 W power supply.
- ▶ Laptop in the same network, with an SSH client.
- ▶ No XIAOML Kit today.

Software on the card

- ▶ Ollama, installed by Labs/hardware/HW-05/.
- ▶ The environment `~/ollama` with the Python packages `ollama` and `pydantic`.
- ▶ The four models of the next frame. The card has them: the lab needs no download.

Lab files	Labs/day10/ on the laptop, <code>~/edgeai/day10/</code> on the board
Commands	On the board, in <code>~/edgeai/day10/</code> , with <code>~/ollama</code> active

The four models of the lab



Model	Type	Download	Used in
llama3.2:1b	text, Q8_0	1.32 GB	Parts A, B, C1
llama3.2:3b	text, Q4_K_M	2.02 GB	Parts A, B, C1
nommic-embed-text	embedding, 768 values	0.27 GB	Part C, option 1
llava-phi3:3.8b	image and text	2.93 GB	Part C, option 2

- ▶ The Python library `ollama` is a client. The model runs in the service (Part 2 of the lecture).
- ▶ The first request loads the model from the card. The scripts do not count this request.

Source: Download sizes: the Ollama library, read on 2026-10-01 ([Labs/hardware/HW-05/](#)).

Part A: run and measure (45 min)

Step	Work	Time
1	Copy the folder, connect, ollama list, one chat with --verbose	10 min
2	Estimate the RAM: the weights and the KV cache (Part 1 of the lecture)	5 min
3	Task A1: the function token_rate in measure_slm.py	5 min
4	Measure the two models	15 min
5	The context and the threads	10 min

```
scp -r day10 edge@pi-NN.local:~/edgeai/      # on the laptop, in Labs/  
ssh edge@pi-NN.local                        # then on the board  
cd ~/edgeai/day10  
source ~/ollama/bin/activate  
ollama list  
ollama run llama3.2:1b --verbose             # then /bye
```

- ▶ ollama list shows the four models of the lab.
- ▶ **You record:** the prompt eval rate and the eval rate of your first question.
- ▶ **Task A1:** tokens / time in seconds. Ollama gives nanoseconds.

```
watch -n 2 vgcncmd measure_temp      # in a second SSH terminal  
python measure_slm.py                # in the first terminal
```

The script uses the method of Day 9:

Part of the method	Value
Not counted	One request that loads the model, one request to warm up
Timed	10 requests: the 5 prompts of prompts.txt, two times
Statistic	The median of the rates, and the lowest rate
Settings	Context 2048 tokens, at most 128 tokens for each answer
Window	The counters of Ollama: the model only

- ▶ The script prints Task A1: complete, then one line for each model:
load s, prompt tk, prompt tk/s, eval tk/s, min tk/s, model MB,
temp C.
- ▶ **You record:** the two lines in the benchmark table.

```
python measure_slm.py --memory-only --models llama3.2:3b --num-ctx 2048
python measure_slm.py --memory-only --models llama3.2:3b --num-ctx 8192
for t in 1 2 4; do
    python measure_slm.py --models llama3.2:1b --runs 1 --num-predict 32 \
        --num-thread $t
done
```

- ▶ `--memory-only` loads the model and prints its memory in ollama ps. Compare the difference with your estimate.
- ▶ The thread runs use short answers, so that 1 thread is not too slow.

Before you run. The KV cache of llama3.2:3b has 114 688 bytes for each token. How many MB more does the model need with 8192 tokens than with 2048 tokens? Is the generation with 4 threads 4 times as fast as with 1 thread? Write your answers in the report.

Part B: structured output (45 min)

Step	Work	Time
1	Task B1: the class <code>Command</code> in <code>part_b/structured.py</code>	10 min
2	Structured output: with and without the schema, two models	15 min
3	Task B2: the function <code>check_call</code> in <code>part_b/tools.py</code>	10 min
4	Function calls with the two models	10 min

```
class Command(BaseModel):          # Task B1: replace each str
    device: str                    # -> Literal["lamp", "fan", ...]
    action: str
    room: str
```

```
python part_b/structured.py          # llama3.2:1b, schema
python part_b/structured.py --no-schema # the prompt only
python part_b/structured.py --model llama3.2:3b # and with --no-schema
```

- ▶ The last line: valid ... of 10, correct ... of 10.
- ▶ **You record:** the four results, and each valid answer with a wrong value.

The model gets two tools: `read_sensor(room)` and `set_heater(room, celsius)`. The sensor values are simulated.

Task B2: the guard. `check_call` returns (True, arguments) only if:

- ▶ the function name is a key of `FUNCTIONS`,
- ▶ the room is one of the four rooms,
- ▶ for `set_heater`: `celsius` is an integer from 5 to 30. The model can send the text "21": convert it.

```
python part_b/tools.py  
python part_b/tools.py --model llama3.2:3b
```

```
Request: Set the heater in the garage to 300 degrees.  
set_heater({'room': 'garage', 'celsius': '300'}) -> REFUSED: celsius 300 ...
```

- ▶ **You record:** each refused call and its reason.

Key point: The model proposes. Your code checks, and then it acts.

Part C: select one application (60 min)

	Option 1: retrieval	Option 2: image description
Script	<code>part_c/rag.py</code>	<code>part_c/describe_image.py</code>
Task	C1: cosine and top_k	C2: the limits of the class Scene
Models	nomic-embed-text, llama3.2:1b and 3b	llava-phi3:3.8b
Test set	5 questions about 12 facts of a fictional greenhouse controller	5 COCO images of kitchens and tables
Output	JSON: <code>answer</code> , <code>fact_number</code>	JSON: <code>caption</code> , <code>objects</code> , <code>containers</code>
Time	seconds for each question	1 to 4 minutes for each image (estimate)

- ▶ The two options end with the line `valid structured output ...` of 5. The check criterion needs 5 of 5.

Source: Time of option 2: an estimate from the published time of one image description (lab "Small Language Models" of *Machine Learning Systems*).

Step	Work	Time
1	Task C1: cosine and top_k	10 min
2	The five test questions with the two models	15 min
3	The number of facts: $k = 1, 2$, and 4	15 min
4	Three facts of your own, and one question with <code>--ask</code>	10 min
5	The questions of Part C in the report	10 min

```
python part_c/rag.py                                # llama3.2:1b, k = 2
python part_c/rag.py --model llama3.2:3b
python part_c/rag.py --k 4
python part_c/rag.py --ask "When does the fan stop?"
```

- ▶ For each question: the facts, the prompt tokens, the time, and the answer with the number of its fact.
- ▶ The check of the script looks only for the number. Read each answer: a valid answer can be wrong.

Step	Work	Time
1	Task C2: at most 5 objects, at most 40 characters each	10 min
2	The five test images	25 min
3	Compare containers with the counts of COCO	10 min
4	The questions of Part C in the report	15 min

```
python part_c/describe_image.py  
python part_c/describe_image.py --image part_c/images/000000166277.jpg
```

- ▶ Without the limits of Task C2, a model can write until the token limit. The JSON is then not complete.
- ▶ `part_c/images/README.md` gives the cups and bottles that the annotators of COCO marked in each image.
- ▶ **You record:** the time, the caption, and the count for each image.

The instructor checks this at the end of the lab:

- ▶ ✓ `python measure_slm.py` prints Task A1: complete.
- ▶ ✓ The benchmark table has the two models with measured numbers: the median generation rate, the prompt rate, the model memory, and the temperature, with the method.
- ▶ ✓ `part_b/structured.py` prints Task B1: complete, and `part_b/tools.py` prints Task B2: complete and REFUSED for the request of 300 degrees.
- ▶ ✓ The application of Part C prints valid structured output 5 of 5 for its five test prompts.
- ▶ ✓ The Decision Log gives numbers and names one trade-off.

Hand in: the file `report.md`.

Write about 100 words. State one design decision, give your measured numbers, and name the trade-off.

A farm wants an offline assistant on a Raspberry Pi 5 in a greenhouse. The workers type questions about the manual of the greenhouse controller. The first word of the answer must appear in less than 10 seconds.

- ▶ Which model and which quantization level do you select?
- ▶ Which context and which k?
- ▶ Which part of the job does not need a language model?

A good Decision Log:

- ▶ gives the rates of your board with their method, and the prompt tokens of one question,
- ▶ names the correct answers of your test, not only the valid ones,
- ▶ names what you lose: for example speed against correct facts.

If a part does not work, and common problems

Problem	Fix or fallback
No module named 'ollama'	<code>source ~/ollama/bin/activate</code>
Failed to connect to Ollama	<code>sudo systemctl start ollama</code>
SKIP llama3.2:3b: not downloaded	Ask the instructor. ollama pull needs the network and some minutes.
The first request needs many seconds	Ollama loads the model. The scripts do not count this request.
The rate falls during a long run	Check the cooler and <code>vcgencmd</code> <code>get_throttled</code> . Write the value in the report.
The board is very slow, <code>free -m</code> shows swap	Use a smaller context or the 1B model
<code>not valid</code> for an answer	The answer reached the token limit. Check the limits of the task.
A valid answer is wrong	A schema makes the output valid, not correct. Write the case in the report.
Option 2 is too slow	Run two images with <code>--image</code> , and write this in the report
A task is not complete in time	Use the file of <code>solutions/</code> , and write this in the report

The README of the lab has the complete list.

This lecture adapts material from these sources:

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.